WEEK 3: ASSIGNMENT REPORT

PROJECT OBJECTIVES

- **Algorithm Mastery:** To implement complex sorting methodologies (Merge Sort, Quick Sort, Heap Sort) without using standard/built-in high-level library functions.
- **Time & Space Efficiency:** To understand practical execution differences when dealing with scale boundaries (N≤100,000 elements).
- **Real-World Data Processing:** Applying mathematical constraints like average filtration, median identification, and percentile performance over unsorted telemetry data.

Primary Topic	Assignment Code & Problem Statement	Complexity Level	Primary Technical Competency Explored
Topic 1: Merge Sort	**Question 1:** Annual Placement Packages Sorting	Easy	Divide and Conquer Strategy, Recursive Splitting, O(NlogN) Efficiency
	Question 2: Commerce Operational Processing Times	Medium	Statistical Analysis, Median Calculations, Handling Voluminous Datasets
Topic 2: Quick Sort	**Question 1:** Cloud Infrastructure Server Latency Sorting	Easy	Pivot Selection, In-Place Partitioning, Memory-Efficient Sorting
	Question 2: Financial Stock Market Trade-Value Analytics	Medium	Descending Order Inversion, Top-K Metric Analysis, Deviation from Averages
Topic 3: Heap Sort	**Question 1:** Online Gaming Tournament Leaderboard	Easy	Binary Tree Representation, Max-Heap Creation, Guaranteed Worst-Case Time Boundary
	Question 2: Multi-Specialty Hospital Emergency Services	Medium	Priority Queue Concepts, Complete Binary Tree Extraction, Percentage-Based Quality Metrics

Week 3

Sorting Algorithms

Topic 1 : Merge Sort

Question 1 (Easy) : University Placement Data Sorting

Problem - The college placement team collected salary data from students, but it is unorganized because entries were made randomly as recieved. We need to sort them in ascending order.

Why Merged sort - Since the data can grow up to thousand of students ($N \leq 100,000$), simple algorithms like Bubble sort will become very slow. Merge sort handles large data efficiently with a guranteed time complexity of $O(N \log N)$.

How does it work - It uses a Divide and Conquer approach. It splits the array of packages into two halves, recursively sorts them, and then merges them back together in a sorted manner.

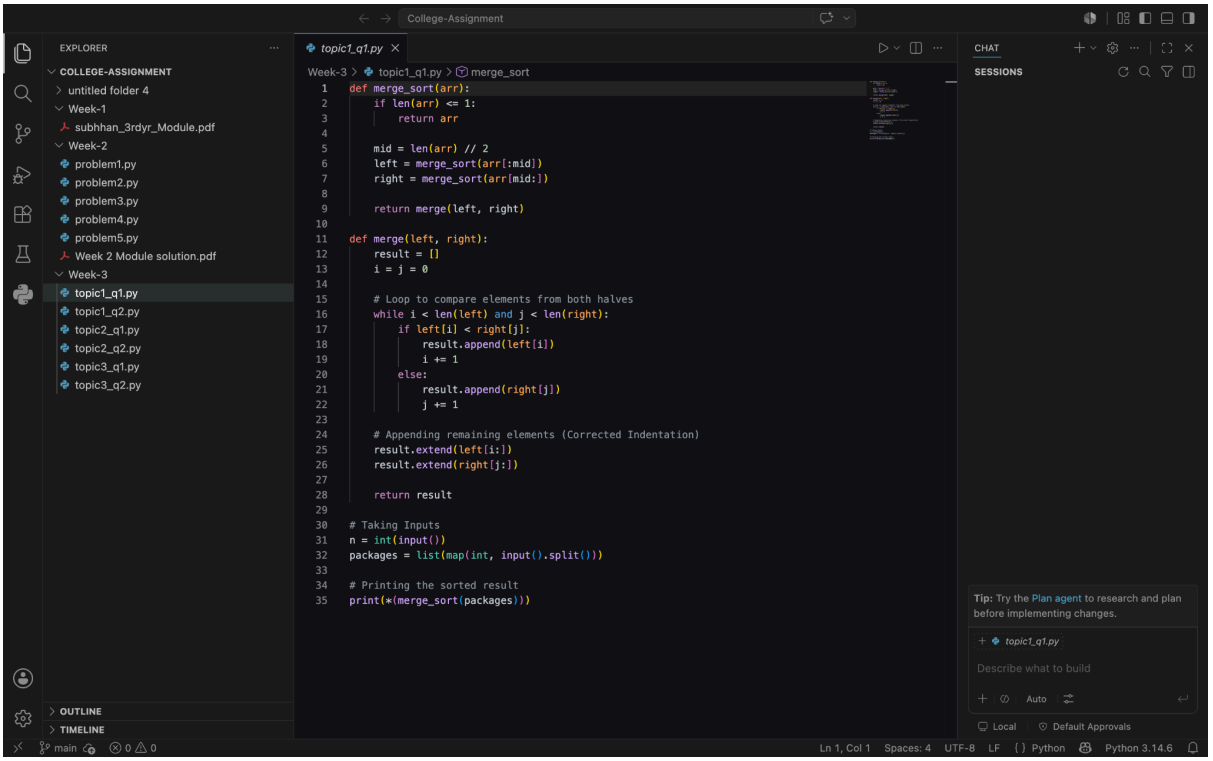
Code Reference - topic1_q1.py

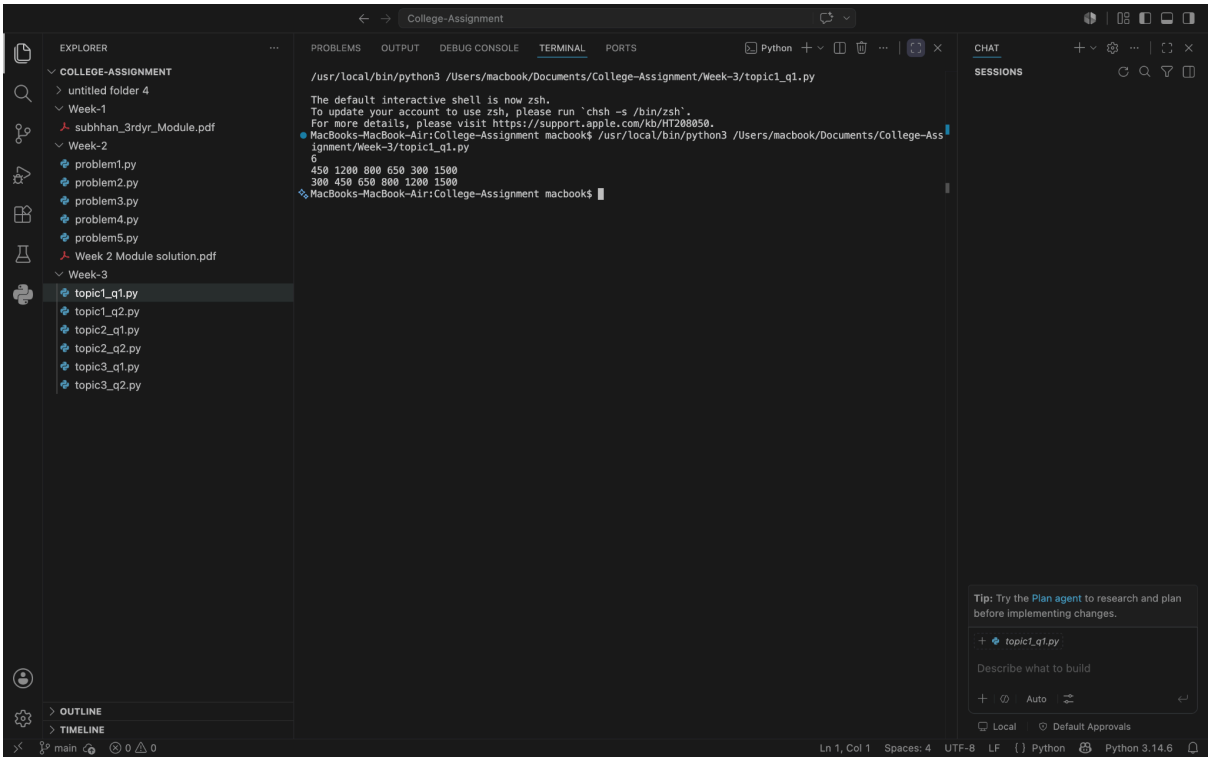
Question 2 (Medium) : E-commerce order processing Times

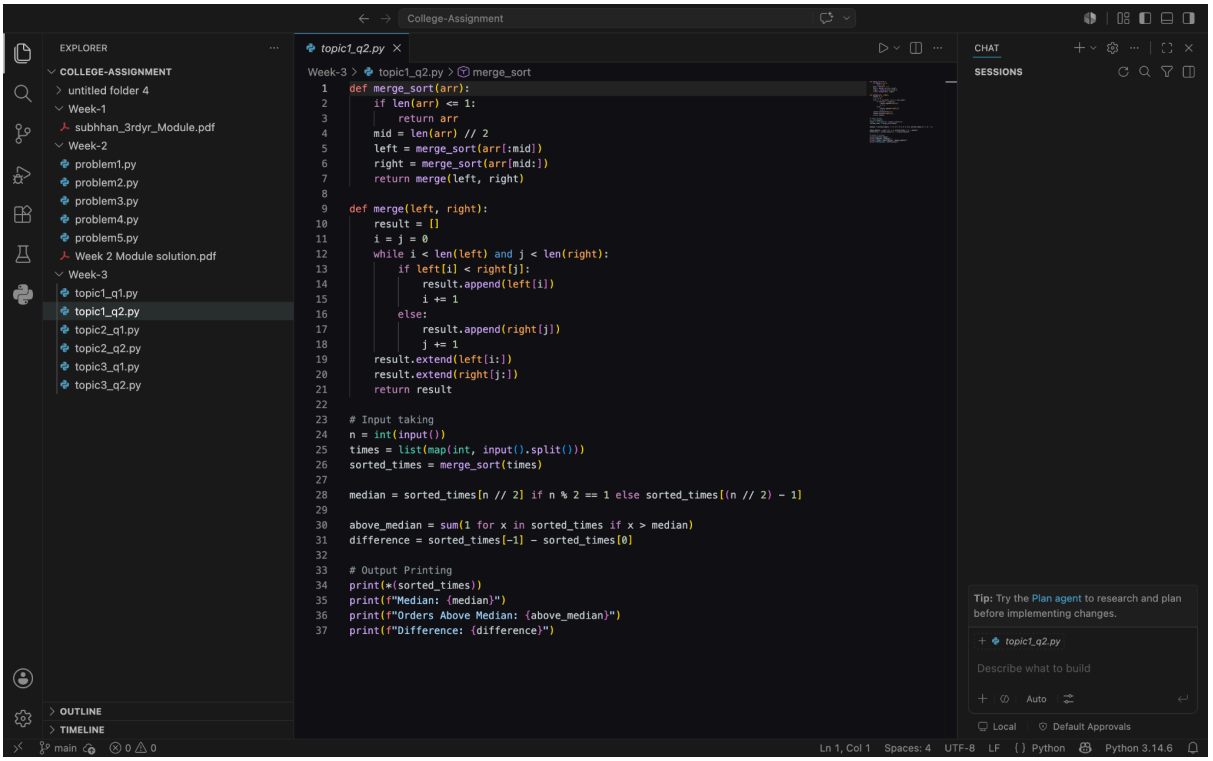
Problem - We need to sort warehouse processing times to study operational efficiency and calculate metrics like median, Order above median, and the time difference between fastest / slowest orders.

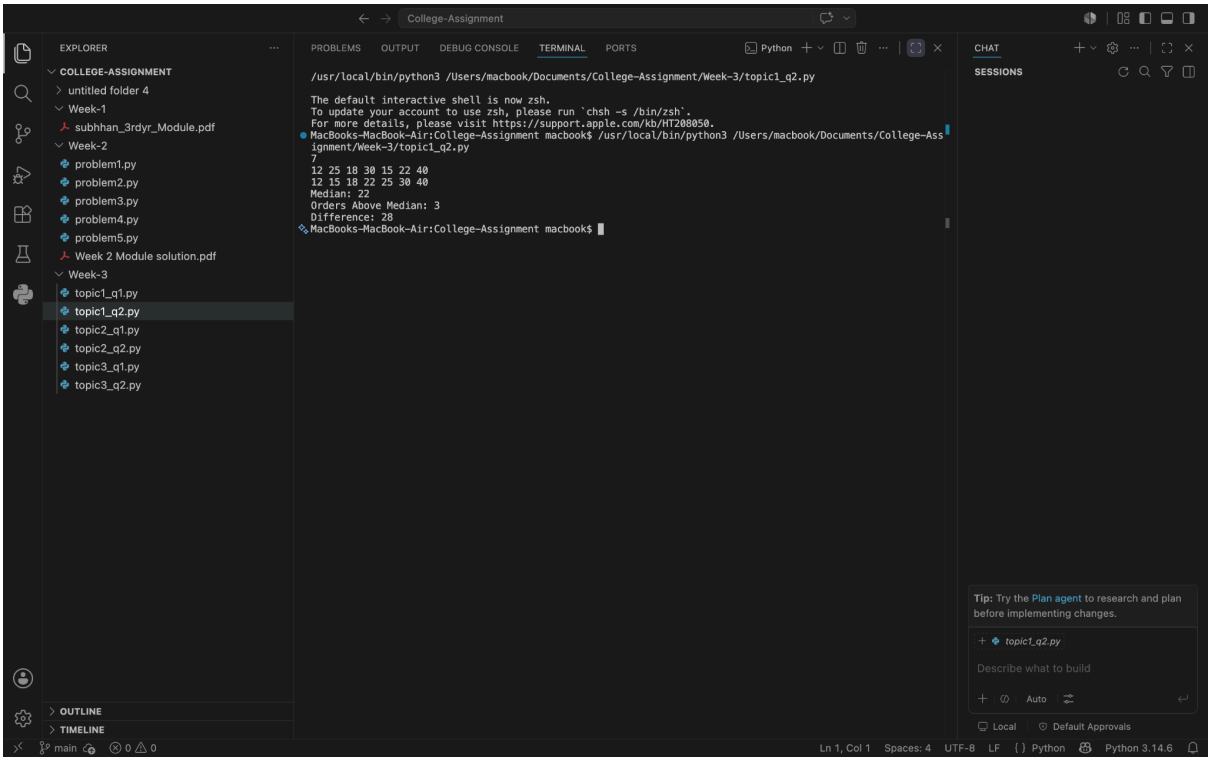
Tricky part (Median and Condition) - Finding the median depend on whether N is odd or even. After sorting. If N is odd, the middle element is the median. If N is even, we pick the element at $(n//2)-1$ as per the standard question logic. To find orders above the median, we loop through the sorted list and count elements strictly greater than it.

Code Reference - topic1_q2.py


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
topic1_q1.py
Week-3 > topic1_q1.py > merge_sort
def merge_sort(arr):
if len(arr) <= 1:
return arr
mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])
return merge(left, right)
def merge(left, right):
result = []
i = j = 0
# Loop to compare elements from both halves
while i < len(left) and j < len(right):
if left[i] < right[j]:
result.append(left[i])
i += 1
else:
result.append(right[j])
j += 1
# Appending remaining elements (Corrected Indentation)
result.extend(left[i:])
result.extend(right[j:])
return result
# Taking Inputs
n = int(input())
packages = list(map(int, input().split()))
# Printing the sorted result
print(*(merge_sort(packages)))
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic1_q1.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
/usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic1_q1.py
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:College-Assignment macbook$ /usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic1_q1.py
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500
MacBooks-MacBook-Air:College-Assignment macbook$
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic1_q1.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
topic1_q2.py
Week-3 > topic1_q2.py > merge_sort
def merge_sort(arr):
if len(arr) <= 1:
return arr
mid = len(arr) // 2
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])
return merge(left, right)
def merge(left, right):
result = []
i = j = 0
while i < len(left) and j < len(right):
if left[i] < right[j]:
result.append(left[i])
i += 1
else:
result.append(right[j])
j += 1
result.extend(left[i:])
result.extend(right[j:])
return result
# Input taking
n = int(input())
times = list(map(int, input().split()))
sorted_times = merge_sort(times)
median = sorted_times[n // 2] if n % 2 == 1 else sorted_times[(n // 2) - 1]
above_median = sum(1 for x in sorted_times if x > median)
difference = sorted_times[-1] - sorted_times[0]
# Output Printing
print(*(sorted_times))
print(f"Median: {median}")
print(f"Orders Above Median: {above_median}")
print(f"Difference: {difference}")
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic1_q2.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
/usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic1_q2.py
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:College-Assignment macbook$ /usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic1_q2.py
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
MacBooks-MacBook-Air:College-Assignment macbook$
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic1_q2.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6

Topic 2- Quick Sort

Question 1 (Easy): Server Response Time sorting

Problem - A cloud company wants to arrange server latency times from fastest to slowest (ascending order) to find lagging server without using any built-in function.

Why Quick sort - Quick sort is an in-place sorting algorithms, meaning it does not required extra memory / arrays like Merge sort, making it highly memory efficient.

How does it work - It picks a middle element as a 'Pivot'. Then, it divides the remaining elements into three groups: items smaller than pivot (left), items equal to pivot (middle), and items larger than pivot (right). It repeats this recursively.

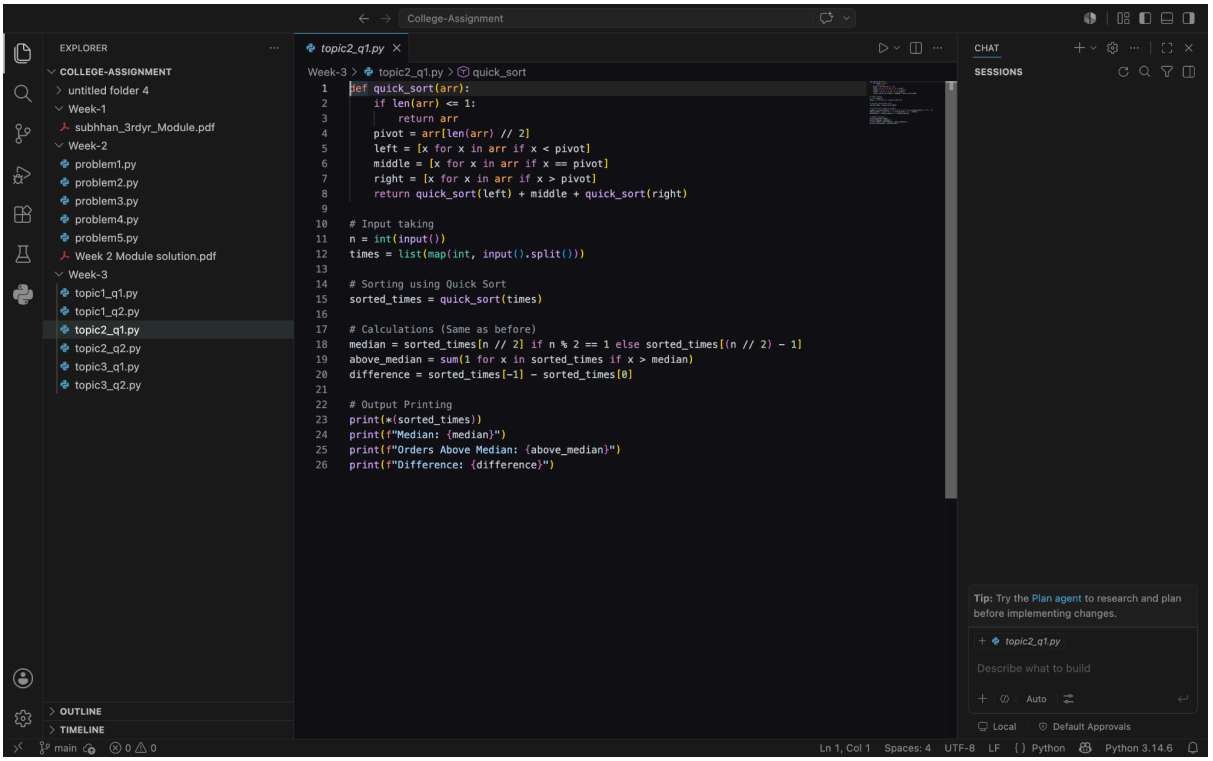
Code Reference - topic2-q1.py

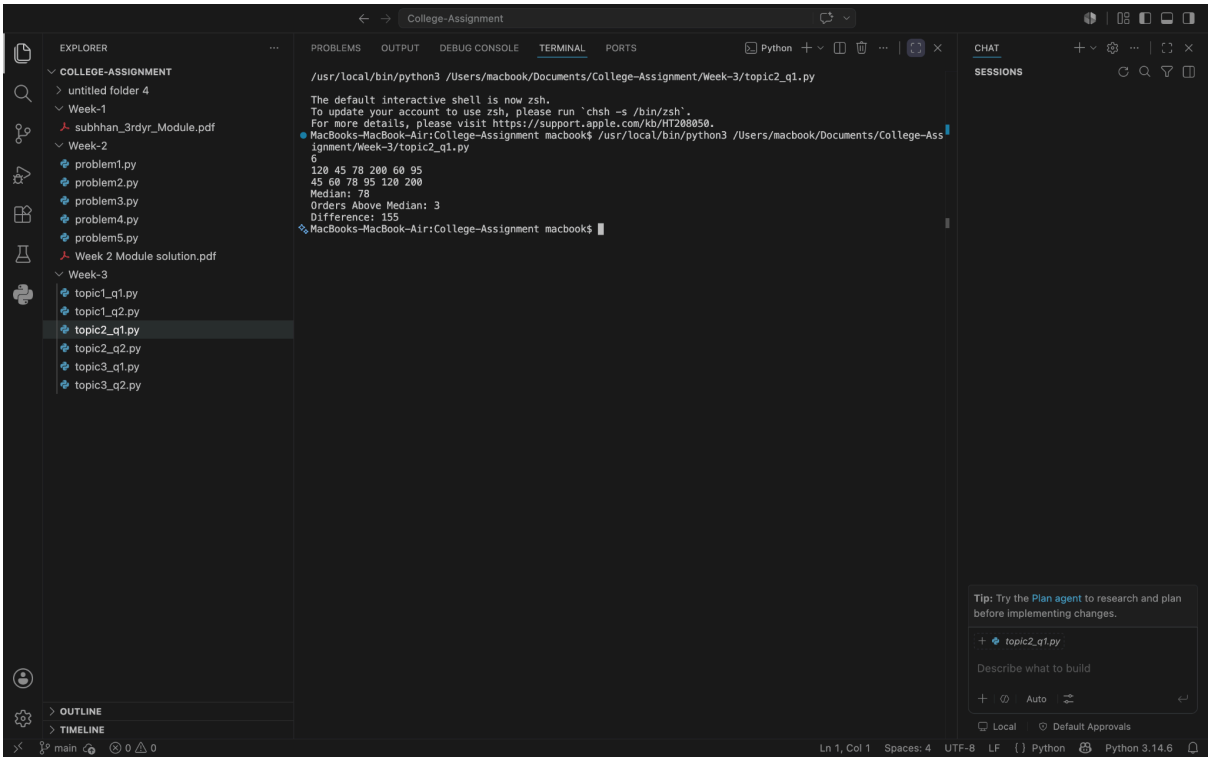
Question 2 (Medium): Stock Exchange Trade values Analysis

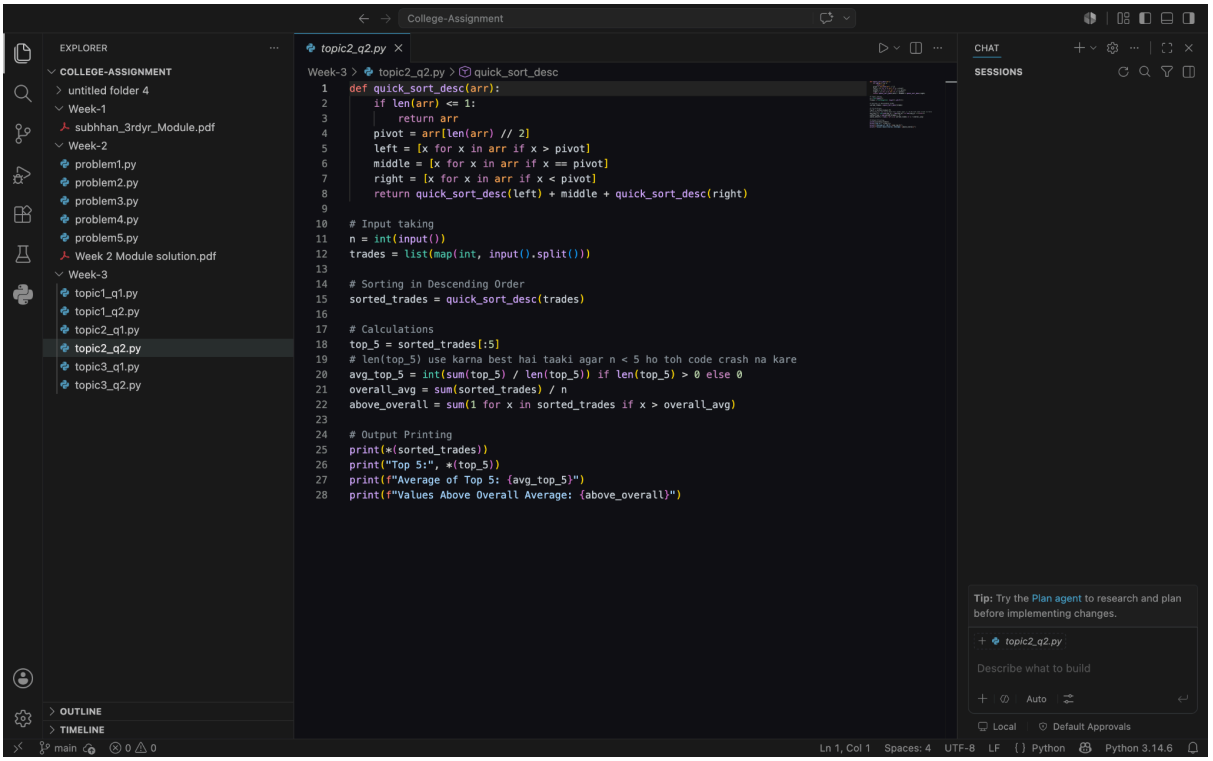
Problem - Financial data (trade data) must be analyzed in descending order (highest to lowest). We need to calculate the average of the top 5 trade and see how many total trade performed better than the overall market average.

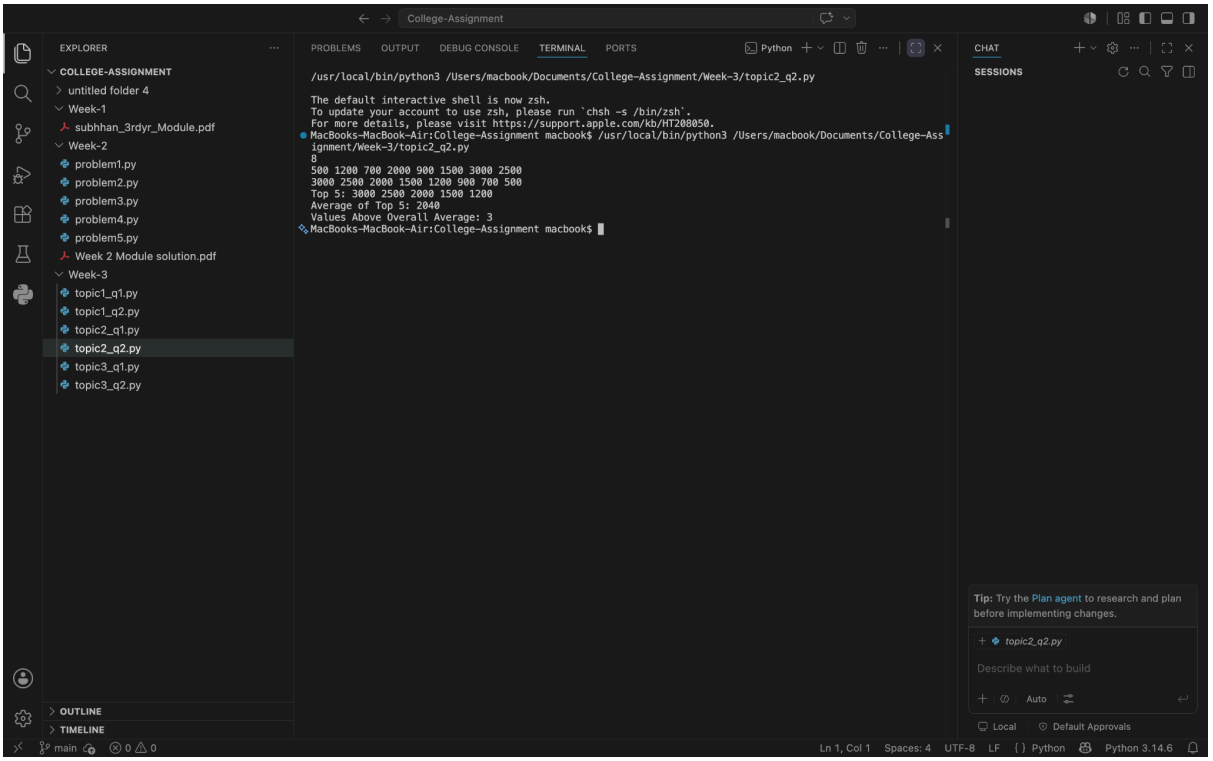
How we changed Quicksort for Descending Order - In standard Quick sort, elements smaller than the pivot go to the left. To sort in descending order, we inverted the conditions: items greater than the pivot go into the left array, and items smaller go to the right array.

Code Reference - topic2-q2.py


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
topic2_q1.py
Week-3 > topic2_q1.py > quick_sort
def quick_sort(arr):
if len(arr) <= 1:
return arr
pivot = arr[len(arr) // 2]
left = [x for x in arr if x < pivot]
middle = [x for x in arr if x == pivot]
right = [x for x in arr if x > pivot]
return quick_sort(left) + middle + quick_sort(right)
# Input taking
n = int(input())
times = list(map(int, input().split()))
# Sorting using Quick Sort
sorted_times = quick_sort(times)
# Calculations (Same as before)
median = sorted_times[n // 2] if n % 2 == 1 else sorted_times[(n // 2) - 1]
above_median = sum(1 for x in sorted_times if x > median)
difference = sorted_times[-1] - sorted_times[0]
# Output Printing
print(*(sorted_times))
print(f"Median: {median}")
print(f"Orders Above Median: {above_median}")
print(f"Difference: {difference}")
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic2_q1.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
/usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic2_q1.py
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:College-Assignment macbook$ /usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic2_q1.py
6
120 45 78 200 60 95
45 60 78 95 120 200
Median: 78
Orders Above Median: 3
Difference: 155
MacBooks-MacBook-Air:College-Assignment macbook$
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic2_q1.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
topic2_q2.py
Week-3 > topic2_q2.py > quick_sort_desc
def quick_sort_desc(arr):
if len(arr) <= 1:
return arr
pivot = arr[len(arr) // 2]
left = [x for x in arr if x > pivot]
middle = [x for x in arr if x == pivot]
right = [x for x in arr if x < pivot]
return quick_sort_desc(left) + middle + quick_sort_desc(right)
# Input taking
n = int(input())
trades = list(map(int, input().split()))
# Sorting in Descending Order
sorted_trades = quick_sort_desc(trades)
# Calculations
top_5 = sorted_trades[:5]
# len(top_5) use karna best hai taaki agar n < 5 ho toh code crash na kare
avg_top_5 = int(sum(top_5) / len(top_5)) if len(top_5) > 0 else 0
overall_avg = sum(sorted_trades) / n
above_overall = sum(1 for x in sorted_trades if x > overall_avg)
# Output Printing
print(*(sorted_trades))
print("Top 5:", *(top_5))
print(f"Average of Top 5: {avg_top_5}")
print(f"Values Above Overall Average: {above_overall}")
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic2_q2.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
/usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic2_q2.py
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:College-Assignment macbook$ /usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic2_q2.py
8
500 1200 700 2000 900 1500 3000 2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
MacBooks-MacBook-Air:College-Assignment macbook$
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic2_q2.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6

Week 3

Sorting Algorithms

Topic 1: Merge Sort

Question 1 (Easy): University Placement Data Sorting

Problem - The college placement team collected salary data from students, but it is unorganized because entries were made randomly as received. We need to sort them in ascending order.

Why Merged sort - Since the data can grow up to thousand of students ($N \leq 100,000$), simple algorithms like Bubble sort will become very slow. Merge sort handles large data efficiently with a guaranteed time complexity of $O(N \log N)$.

How does it work - It uses a Divide and Conquer approach. It splits the array of packages into two halves, recursively sorts them, and then merges them back together in a sorted manner.

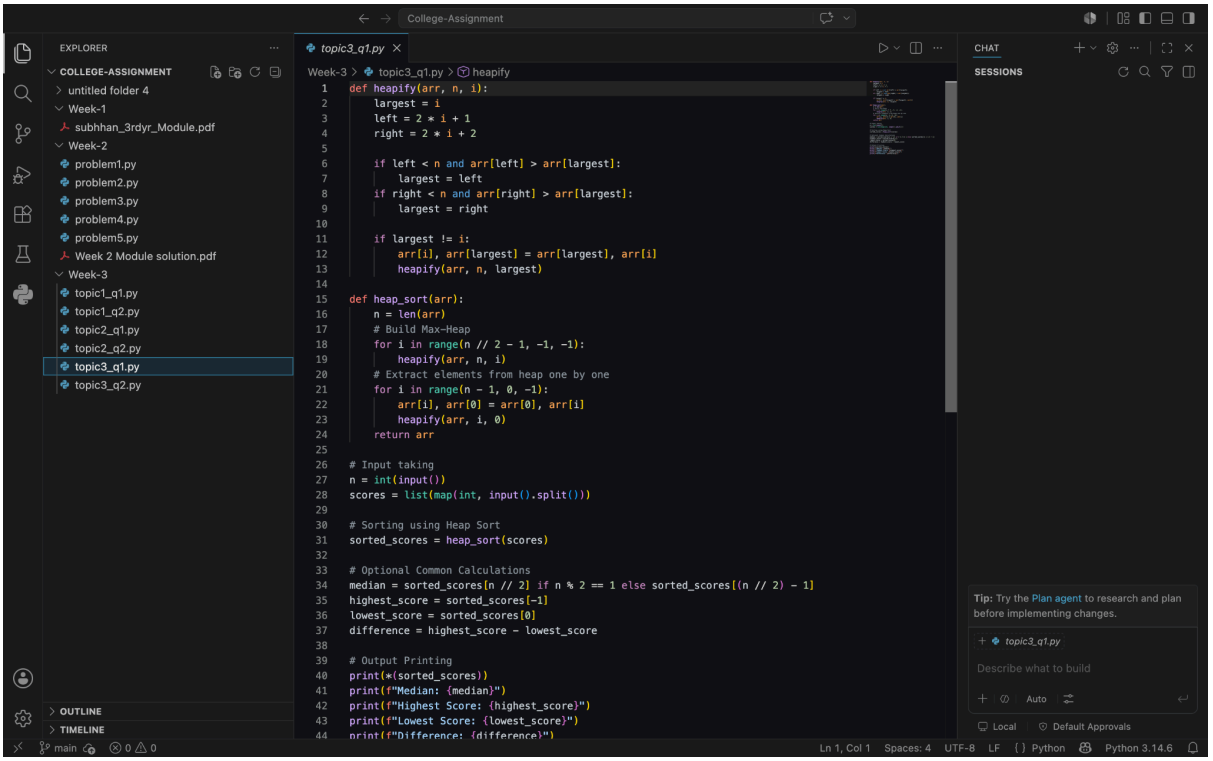
Code Reference - topic1_q1.py

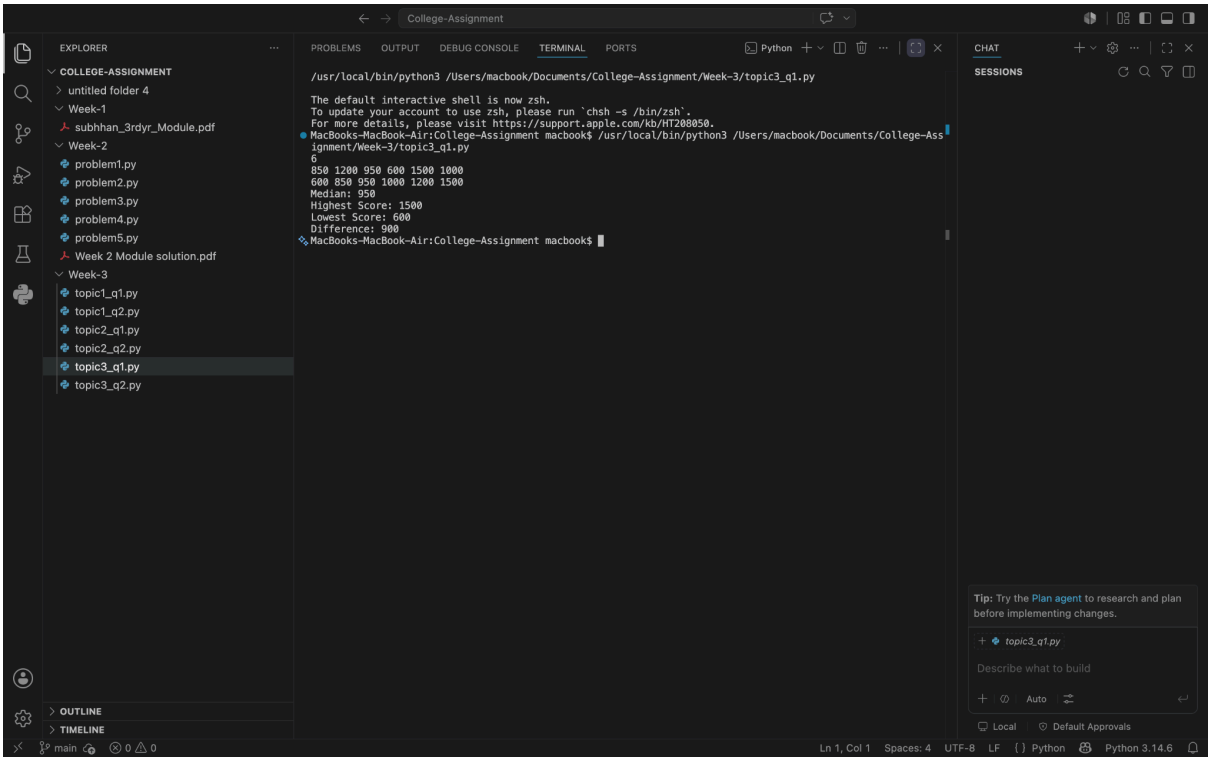
Question 2 (Medium): E-commerce Order processing Times

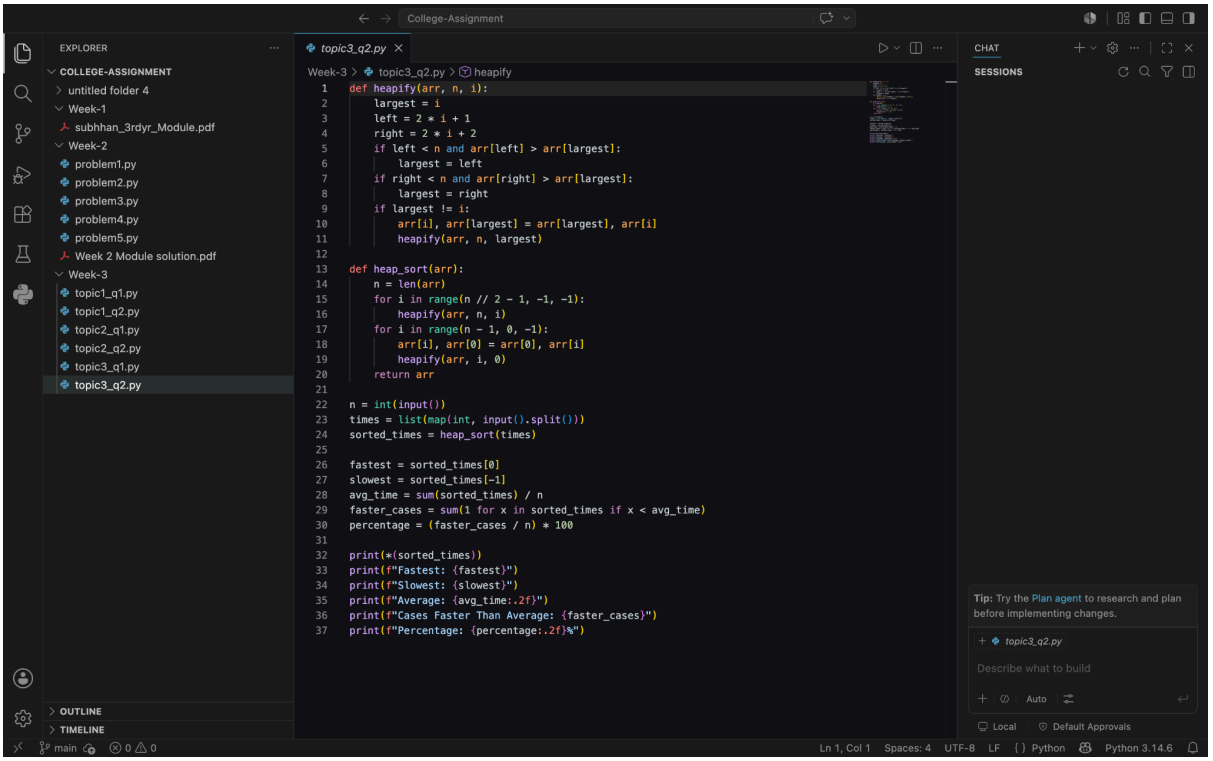
Problem - We need to sort warehouse processing times to study operational efficiency and calculate metrics like median, Orders above median, and the time difference between fastest / slowest orders.

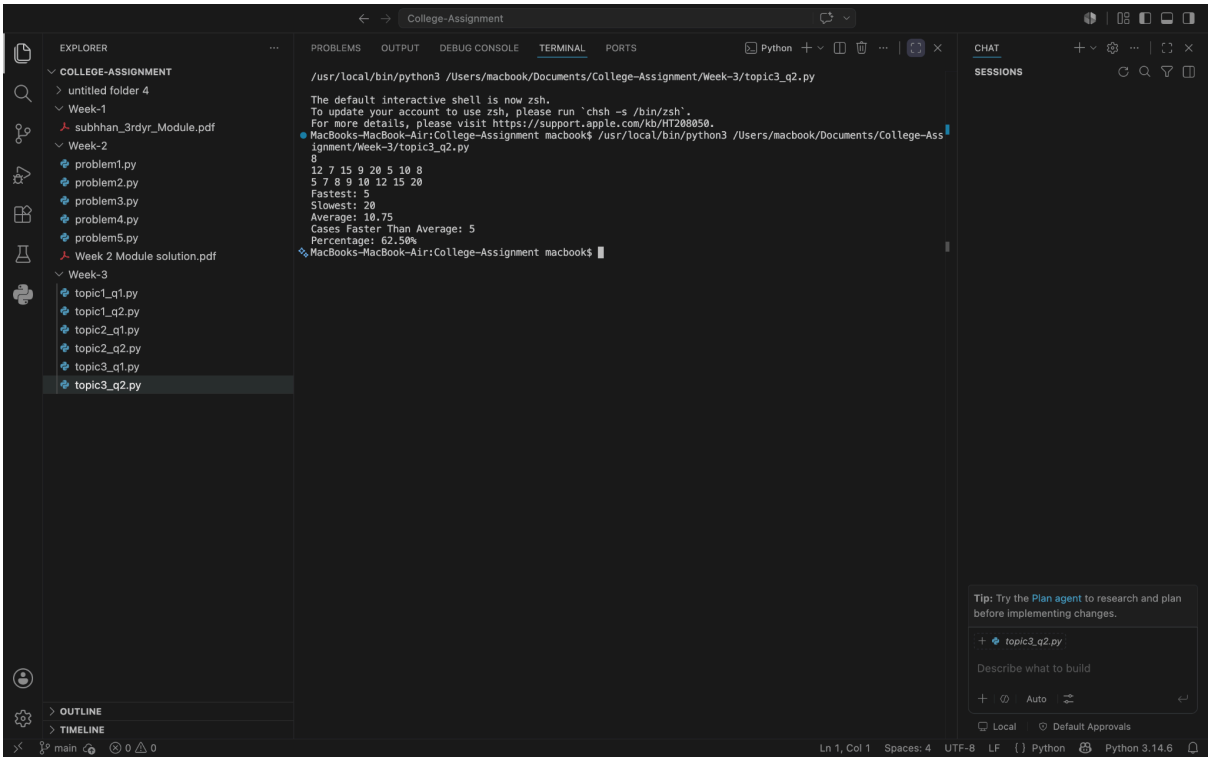
Tricky part (Median and Condition) - Finding the median depend on whether N is odd or even. After sorting. If N is odd, the middle element is the median. If N is even, we pick the element at $(n//2)-1$ as per the standard question logic. To find orders above the median, we loop through the sorted list and count elements strictly greater than it.

Code Reference - topic1_q2.py


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
topic3_q1.py
Week-3 > topic3_q1.py > heapify
def heapify(arr, n, i):
largest = i
left = 2 * i + 1
right = 2 * i + 2
if left < n and arr[left] > arr[largest]:
largest = left
if right < n and arr[right] > arr[largest]:
largest = right
if largest != i:
arr[i], arr[largest] = arr[largest], arr[i]
heapify(arr, n, largest)
def heap_sort(arr):
n = len(arr)
# Build Max-Heap
for i in range(n // 2 - 1, -1, -1):
heapify(arr, n, i)
# Extract elements from heap one by one
for i in range(n - 1, 0, -1):
arr[i], arr[0] = arr[0], arr[i]
heapify(arr, i, 0)
return arr
# Input taking
n = int(input())
scores = list(map(int, input().split()))
# Sorting using Heap Sort
sorted_scores = heap_sort(scores)
# Optional Common Calculations
median = sorted_scores[n // 2] if n % 2 == 1 else sorted_scores[(n // 2) - 1]
highest_score = sorted_scores[-1]
lowest_score = sorted_scores[0]
difference = highest_score - lowest_score
# Output Printing
print(*(sorted_scores))
print(f"Median: {median}")
print(f"Highest Score: {highest_score}")
print(f"Lowest Score: {lowest_score}")
print(f"Difference: {difference}")
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic3_q1.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
/usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic3_q1.py
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:College-Assignment macbook$ /usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic3_q1.py
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500
Median: 950
Highest Score: 1500
Lowest Score: 600
Difference: 900
MacBooks-MacBook-Air:College-Assignment macbook$
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic3_q1.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
topic3_q2.py
Week-3 > topic3_q2.py > heapify
def heapify(arr, n, i):
largest = i
left = 2 * i + 1
right = 2 * i + 2
if left < n and arr[left] > arr[largest]:
largest = left
if right < n and arr[right] > arr[largest]:
largest = right
if largest != i:
arr[i], arr[largest] = arr[largest], arr[i]
heapify(arr, n, largest)
def heap_sort(arr):
n = len(arr)
for i in range(n // 2 - 1, -1, -1):
heapify(arr, n, i)
for i in range(n - 1, 0, -1):
arr[i], arr[0] = arr[0], arr[i]
heapify(arr, i, 0)
return arr
n = int(input())
times = list(map(int, input().split()))
sorted_times = heap_sort(times)
fastest = sorted_times[0]
slowest = sorted_times[-1]
avg_time = sum(sorted_times) / n
faster_cases = sum(1 for x in sorted_times if x < avg_time)
percentage = (faster_cases / n) * 100
print(*(sorted_times))
print(f"Fastest: {fastest}")
print(f"Slowest: {slowest}")
print(f"Average: {avg_time:.2f}")
print(f"Cases Faster Than Average: {faster_cases}")
print(f"Percentage: {percentage:.2f}%")
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic3_q2.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6


College-Assignment
EXPLORER
COLLEGE-ASSIGNMENT
untitled folder 4
Week-1
subhhan_3rdyr_Module.pdf
Week-2
problem1.py
problem2.py
problem3.py
problem4.py
problem5.py
Week 2 Module solution.pdf
Week-3
topic1_q1.py
topic1_q2.py
topic2_q1.py
topic2_q2.py
topic3_q1.py
topic3_q2.py
OUTLINE
TIMELINE
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Python
/usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic3_q2.py
The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:College-Assignment macbook$ /usr/local/bin/python3 /Users/macbook/Documents/College-Assignment/Week-3/topic3_q2.py
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
MacBooks-MacBook-Air:College-Assignment macbook$
CHAT
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
topic3_q2.py
Describe what to build
Auto
Local
Default Approvals
main
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.14.6